

Windows Internals and Kernel Driver
Development
WinDBG Introduction 101
Laboratory Report

Vlad Bartolomei

14 March 2026

Contents

1	Level 1: WinDBG Cheat sheet of most used commands	2
1.1	.symfix – Set Symbol Store Path	2
1.2	.reload (Reload Module)	3
1.3	dt _EPROCESS – TL;DR: This dumps EPROCESS	5
1.4	dt _EPROCESS -v	7
1.5	dt _EPROCESS -b	7
1.6	dt _ETHREAD -v -b	8
1.7	!process	9
1.8	!process 0 0	11
1.9	!process -1 7	12
1.10	!thread	13
1.11	!stacks	13
1.12	!stacks 2	15
1.13	g	16
1.14	.cls	16

1 Level 1: WinDBG Cheat sheet of most used commands

1.1: Very important symbols

There are two important symbols: bang and dot.

- **Bang** ! – execute what you select
- **Dot** . – select

1.1 .symfix – Set Symbol Store Path

Purpose: auto sets the symbol path to point to the Microsoft symbol store. **Checking:** `.sympath` displays what you've just set.

In the context of Windows Drivers, **symbol** = named unit of data or code from a source file that appears in a module. Information about symbols can include:

- name
- type if applicable
- address or register where it is stored
- any parent of child symbols

E.g. Symbols may be **variables** (local and global), **functions**, **any entry point into a module**.

The engine uses symbol information to help interpret data and code in the target. With this information, the engine can search for symbols by name or location in memory and provide a description of a symbol.

Flavours:

- `.symfix+` appends the symbol store path to the existing symbol store path. Without `+`, the existing symbol path is replaced;
- `.symfix[+] LocalSymbolCache` – the latter parameter specifies the directory to be used as a local symbol cache. If the directory does not exist, it will be created. If it is not specified then the sym subdirectory of the debugger installation directory will be used;

```
3: kd> .sympath
Symbol search path is: c:\someSymbols
Expanded Symbol search path is: c:\somesymbols
3: kd> .symfix+ c:\myCache
3: kd> .sympath
Symbol search path is: c:\someSymbols;srv*
Expanded Symbol search path is: c:\somesymbols;cache*c:\myCache;SRV*https://msdl.microsoft.com/download/symbols
```

Execution

```
1: kd> .symfix
1: kd> .symfix
1: kd> .sympath
Symbol search path is: srv*
Expanded Symbol search path is: cache*;SRV*https://msdl.microsoft.com/download/symbols

***** Path validation summary *****
Response                                Time (ms)    Location
Deferred
1: kd> .symfix+ c:\myCache
1: kd> .sympath
Symbol search path is: srv*
Expanded Symbol search path is: SRV*c:\myCache*https://msdl.microsoft.com/download/symbols

***** Path validation summary *****
Response                                Time (ms)    Location
Deferred
```

Figure 1: `.symfix` sets, `.sympath` displays. When examining expanded symbol path, note that `*` is the separating symbol, as `:` would be for `PATH` variable.

1.2 `.reload` (Reload Module)

Purpose: deletes all symbol information for specified module and reloads these symbols as needed. In some cases, this command also reloads or unloads the module itself.

The `.reload` command tells WinDbg to do its initial search to find and load symbol files.

Flavours:

Any of the following options:

/d	Reload all modules in the debugger's module list. (When you omit all parameters, this situation is the default during user-mode debugging.)
/f	Forces the debugger to immediately load the symbols. This parameter overrides <i>lazy symbol loading</i> . For more information, see the following Remarks section.
/i	Ignores a mismatch in the .pdb file versions. (If you do not include this parameter, the debugger does not load mismatched symbol files.) When you use /i , /f is used also, even if you do not explicitly specify it.
/l	Lists the modules but does not reload their symbols. (In kernel mode, this parameter gives the same output as the !drivers extension.)
/n	Reloads kernel symbols only. This parameter does not reload any user symbols. (You can use this option only during kernel-mode debugging.)
/o	Forces the cached files in a symbol server's downstream store to be overwritten. When you use this flag, you should also include /f . By default, the downstream store files are never overwritten. Because the symbol server uses distinct file names for the symbols of every different build of a binary, you do not have to use this option unless you believe your downstream store has become corrupted.
/s	Reloads all modules in the system's module image list. (When you omit all parameters, this situation is the default during kernel-mode debugging.) If you are loading an individual system module by name while you perform user-mode debugging, you must include /s .
/u	Unloads the specified module and all its symbols. The debugger unloads any loaded module whose name matches <i>Module</i> , regardless of the full path. Image names are also searched. For more information, see the note in the following Remarks section.
/unl	Reloads symbols based on the image information in the unloaded module list.
/user	Reloads user symbols only. (You can use this option only during kernel-mode debugging.)
/v	Turns on verbose mode.
/w	Treats <i>Module</i> as a literal string. This treatment prevents the debugger from expanding wildcard characters.

Figure 2: .reload options

Module – Specifies the name of an image on the target system for which to reload symbols on the host computer. Module should include the name and file name extension of the file. Unless you use the /w option, Module might contain a variety of wildcard characters and specifiers. For more information about the syntax, see String Wildcard Syntax. If you omit Module, the behavior of the .reload command depends on which Options you use.

Address – Specifies the base address of the module. Typically, you have to have this address only if the image header has been corrupted or is paged out.

Size – Specifies the size of the module image. In many situations, the debugger knows the correct size of the module. When the debugger does not know the correct size, you should specify Size. This size can be the actual module size or a larger number, but the size should not be a smaller number. Typically, you have to have this size only if the image header has been corrupted or is paged out.

Timestamp – Specifies the timestamp of the module image. In many situations, the debugger knows the correct timestamp of the module. When the

debugger does not know the timestamps, you should specify Timestamp. Typically, you have to have this timestamp only if the image header has been corrupted or is paged out.

1.1: Note

There must be no blank space between the Address, Size, and Timestamp parameters.

```
1: kd> .reload /u ntdll.dll
Unable to find module 'ntdll.dll'
1: kd> .reload
Connected to Windows 10 26100 x64 target at (Sat Mar 14 12:09:09.120 2026 (UTC + 2:00)), ptr64 TRUE
Loading Kernel Symbols
.....
.....
Loading User Symbols

Loading unloaded module list
.....
```

Figure 3: Execution

```
1: kd> .reload -l
Connected to Windows 10 26100 x64 target at (Sat Mar 14 12:11:57.723 2026 (UTC + 2:00)), ptr64 TRUE
System Driver and Image Summary
Base          Code Size    Data Size    Image Name      Creation Time
fffff804`7ca00000 a92000 (10824 k) 1e6000 (1944 k) ntoskrnl.exe    ***** Invalid
fffff804`7e200000      0 ( 0 k) 5000 ( 20 k) hal.dll         ***** Invalid
fffff804`0e450000 15000 ( 84 k) 33000 (204 k) kdnet.dll       ***** Invalid
fffff804`0e3e0000 40000 ( 256 k) 25000 (148 k) kd_02_8086.dll  Wed Jan 19 23:18:46 2000
fffff804`0cc40000  400 ( 1 k) 36a600 (3498 k) mcupdate_GenuineIntel.dll Thu Feb 1 23:57:52 2024
fffff804`0e530000  3000 ( 12 k) 8000 ( 32 k) symcryptk.dll  ***** Invalid
fffff804`0e600000 9d000 ( 628 k) 33000 (204 k) cng.sys        Fri Aug 28 09:08:15 1992
fffff804`0e570000 6b000 ( 428 k) 1e000 (120 k) CLFS.SYS       ***** Invalid
fffff804`0e540000 1c000 ( 112 k) d000 ( 52 k)  tm.sys        Tue Dec 1 15:17:10 1987
fffff804`0e6e0000  b000 ( 44 k) 9000 ( 36 k) winaccel.sys   ***** Invalid
fffff804`0e700000  b000 ( 44 k) f000 ( 60 k) PSHED.dll      ***** Invalid
fffff804`0e720000 5000 ( 20 k) 7000 ( 28 k) BOOTVID.dll   Sun Aug 20 11:48:22 2028
```

Figure 4: Execution

1.3 dt _EPROCESS – TL;DR: This dumps EPROCESS

1.1: dt (Display Type)

This command displays information about a local/global variable or data type. This can display information about both simple data types and structures and unions.

1.2: _EPROCESS

The EPROCESS structure is an opaque structure that serves as the process object for a process. This object resides in the Kernel only.

Syntax UM:

```
dt [-DisplayOpts] [-SearchOpts] [module!]Name [[-SearchOpts] Field]
[Address] [-l List]
```

Syntax KM:

[Processor] dt... and all the rest
Processor can be selected only in kernel mode.

Flavours:

Module – An optional parameter specifying **the module that defines this structure**. If there is a *local variable or type with the same name as a global variable or type*, **you should include module to specify that you mean the global variable**. Otherwise, the dt command will display the local variable, even if the local variable is a case-insensitive match and the global variable is a case-sensitive match.

Name – Specifies the name of a type or global variable. If Name ends with an asterisk (*), a list of all matches is displayed. *Thus, dt A* will list all data types, globals, and statics beginning with "A", but will not display the actual instances of these types.* (If the -v display option is used at the same time, all symbols will be displayed – not just those with associated type information.) You can also replace Name with a period (.) to signify that you want to repeat the most recently used value of Name.

If Name contains a space, it should be enclosed in parentheses. **E.g.** Name = (namePart1 namePart2).

Field – Specifies the field(s) to be displayed. If Field is omitted, all fields are displayed. If Field is followed by a period (.), the first-level subfields of this field will be displayed as well. If Field is followed with a series of periods, the subfields will be displayed to a depth equal to the number of periods. Any field name followed by a period will be treated as a prefix match, as if the -y search option was used. If Field is followed by an asterisk (*), it is treated as only the beginning of the field, not necessarily the entire field, and all matching fields are displayed.

Address – Specifies the address of the structure to be displayed. **If Name is omitted, Address must be included and must specify the address of a global variable.** Address is taken to be a virtual address unless otherwise specified. Use the -p option to specify a physical address. Use an "at" sign (@) to specify a register (for example, @eax).

List – Specifies the field name that links a linked list. The Address parameter must be included.

Execution:

```

1: kd> dt _EPROCESS
nt!_EPROCESS
+0x000 Pcb : _KPROCESS
+0x1c8 ProcessLock : _EX_PUSH_LOCK
+0x1d0 UniqueProcessId : Ptr64 Void
+0x1d8 ActiveProcessLinks : _LIST_ENTRY
+0x1e8 RundownProtect : _EX_RUNDOWN_REF
+0x1f0 Flags2 : Uint4B
+0x1f0 JobNotReallyActive : Pos 0, 1 Bit
+0x1f0 AccountingFolded : Pos 1, 1 Bit
+0x1f0 NewProcessReported : Pos 2, 1 Bit
+0x1f0 ExitProcessReported : Pos 3, 1 Bit
+0x1f0 ReportCommitChanges : Pos 4, 1 Bit
+0x1f0 LastReportMemory : Pos 5, 1 Bit
+0x1f0 ForceWakeCharge : Pos 6, 1 Bit

```

Figure 5: The list is much more extensive.

1.4 dt _EPROCESS -v

dt *Name* -v

Verbose output. Provides additional information:

- total size of a structure
- number of its elements

When this is used along with the -y search option, all symbols are displayed, even those with no associated type information.

Execution:

```

0: kd> dt _EPROCESS -v
nt!_EPROCESS
struct _EPROCESS, 261 elements, 0x840 bytes
+0x000 Pcb : struct _KPROCESS, 65 elements, 0x1c8 bytes
+0x1c8 ProcessLock : struct _EX_PUSH_LOCK, 7 elements, 0x8 bytes
+0x1d0 UniqueProcessId : Ptr64 to Void
+0x1d8 ActiveProcessLinks : struct _LIST_ENTRY, 2 elements, 0x10 bytes
+0x1e8 RundownProtect : struct _EX_RUNDOWN_REF, 2 elements, 0x8 bytes
+0x1f0 Flags2 : Uint4B
+0x1f0 JobNotReallyActive : Bitfield Pos 0, 1 Bit
+0x1f0 AccountingFolded : Bitfield Pos 1, 1 Bit

```

Figure 6: Compared to image 5 this command lists additional data (no. elements, size in bytes).

1.5 dt _EPROCESS -b

dt *Name* -b

Displays **blocks** recursively. If a displayed structure contains substructures, it is expanded recursively to arbitrary depths and displayed in full. Pointers are expanded only if they are in the original structure, not in substructures.

Execution:

```

0: kd> dt _EPROCESS -b
nt!_EPROCESS
+0x000 Pcb                : _KPROCESS
+0x000 Header              : _DISPATCHER_HEADER
+0x000 Lock                : Int4B
+0x000 LockNV              : Int4B
+0x000 Type                : Uchar
+0x001 Signalling          : Uchar
+0x002 Size                : Uchar
+0x003 Reserved1           : Uchar
+0x000 TimerType           : Uchar
+0x001 TimerControlFlags   : Uchar
+0x001 Absolute            : Pos 0, 1 Bit
+0x001 Wake                : Pos 1, 1 Bit
+0x001 EncodedTolerableDelay : Pos 2, 6 Bits

```

Figure 7: Pcb structure expanded

```

+0x1c8 ProcessLock        : _EX_PUSH_LOCK
+0x000 Locked             : Pos 0, 1 Bit
+0x000 Waiting            : Pos 1, 1 Bit
+0x000 Waking             : Pos 2, 1 Bit
+0x000 MultipleShared     : Pos 3, 1 Bit
+0x000 Shared             : Pos 4, 60 Bits
+0x000 Value              : Uint8B
+0x000 Ptr                : Ptr64
+0x1d0 UniqueProcessId    : Ptr64
+0x1d8 ActiveProcessLinks : _LIST_ENTRY
+0x000 Flink              : Ptr64

```

Figure 8: ProcessLock and ActiveProcessLinks expanded, UniqueProcessId listed

Compared to image 5, we can see in images 7 and 8 that option `-b` expands the structures where is the case. In 5 Pcb, ProcessLock, UniqueProcessId and ActiveProcessLinks were only listed, whereas in 7 and 8 they are expanded recursively.

1.6 dt _ETHREAD -v -b

1.3: _ETHREAD

The ETHREAD structure is an opaque structure that serves as the thread object for a thread. This object resides in the Kernel only.

Execution:


```

0: kd> dt _ETHREAD /b /v
nt!_ETHREAD
struct _ETHREAD, 149 elements, 0x798 bytes
+0x000 Tcb : struct _KTHREAD, 237 elements, 0x4c0 bytes
+0x000 Header : struct _DISPATCHER_HEADER, 59 elements, 0x18 bytes
+0x000 Lock : Int4B
+0x000 LockNV : Int4B
+0x000 Type : UChar
+0x001 Signalling : UChar
+0x002 Size : UChar
+0x003 Reserved1 : UChar
+0x000 TimerType : UChar
+0x001 TimerControlFlags : UChar
+0x001 Absolute : Bitfield Pos 0, 1 Bit
+0x001 Wake : Bitfield Pos 1, 1 Bit
+0x001 EncodedTolerableDelay : Bitfield Pos 2, 6 Bits
+0x002 Hand : UChar
+0x003 TimerMiscFlags : UChar
+0x003 Index : Bitfield Pos 0, 6 Bits

```

Figure 9: Enter Caption

```

+0x098 ApcStateFill : (43 elements) UChar
+0x0c3 Priority : Char
+0x0c4 UserIdealProcessor : Uint4B
+0x0c8 WaitStatus : Int8B
+0x0d0 WaitBlockList : Ptr64 to
+0x0d8 WaitListEntry : struct _LIST_ENTRY, 2 elements, 0x10 bytes
+0x000 Flink : Ptr64 to
+0x008 Blink : Ptr64 to
+0x0d8 SwapListEntry : struct _SINGLE_LIST_ENTRY, 1 elements, 0x8 bytes
+0x000 Next : Ptr64 to
+0x0e8 Queue : Ptr64 to
+0x0f0 Teb : Ptr64 to
+0x0f8 RelativeTimerBias : Uint8B
+0x100 Timer : struct _KTIMER, 7 elements, 0x40 bytes
+0x000 Header : struct _DISPATCHER_HEADER, 59 elements, 0x18 bytes
+0x000 Lock : Int4B
+0x000 LockNV : Int4B
+0x000 Type : UChar
+0x001 Signalling : UChar

```

Figure 10: More interesting fields; we have confirmation that TEB is accessible to ETHREAD by a pointer; we can also see fields regarding event-based thread execution (see last homework)

1.7 !process

This is an extension.

Purpose: displays information about the specified process, or about all processes, including the EPROCESS block. Usable only on kernel-mode debugging (which we have with windbg, because we attached to the kernel).

Syntax:

!process [/s *Session*] [/m *Module*] [*Process* [*Flags*]] (1)

!process [/s *Session*] [/m *Module*] 0 *Flags* *ImageName* (2)

Flavours:

/s *Session* – Specifies the session that owns the desired process.

/m *Module* – Specifies the module that owns the desired process.

Process – Specifies the hexadecimal address or the process ID of the process on the target computer.

The value of *Process* determines *whether* the !process extension *displays a process address or a process ID*. If *Process* is omitted in any version of Windows,

the debugger displays data only about the current system process. **If Process is 0 and ImageName is omitted, the debugger displays information about all active processes. If -1 is specified for Process information about the current process is displayed.**

Flags – Specifies the level of detail to display. Flags can be any combination of the following bits.

- If Flags is 0, only a minimal amount of information is displayed.
- the default (varies according to the version of Windows and the value of Process) is 0x3
 - if Process is omitted
 - or if the Process is either 0 or -1
- otherwise, the default is 0xF.

Bit 0 (0x1) – Displays **time and priority statistics**.

Bit 1 (0x2) – Displays a **list of threads and events** associated with the process, and **their wait states**.

Bit 2 (0x4) – Displays a **list of threads** associated with the process. If this is included without Bit 1 (0x2), each thread is displayed on a single line. *If this is included along with Bit 1, each thread is displayed with a stack trace.*

Bit 3 (0x8) – (Windows XP and later) Displays **the return address, the stack pointer, and (on Itanium-based systems) the bsp register value for each function**. The display of function arguments is suppressed.

Bit 4 (0x10) – (Windows XP and later) **Sets the process context equal to the specified process for the duration of this command. This results in a more accurate display of thread stacks**. Because this flag is equivalent to using .process /p /r for the specified process, any existing user-mode module list will be discarded. If Process is zero, the debugger displays all processes, and the process context is changed for each one. If you are only displaying a single process and its user-mode state has already been refreshed (for example, with .process /p /r), it is not necessary to use this flag. This flag is only effective when used with Bit 0 (0x1).

ImageName – Specifies the name of the process to be displayed. The debugger displays all processes whose executable image names match ImageName. The image name must match that in the EPROCESS block. When ImageName is specified, Process must be zero.

Execution:

1.2: CID

That is System PID.

```

2: kd> !process
PROCESS fffff8182266a3040
  SessionId: none Cid: 0004 Peb: 00000000 ParentCid: 0000
  DirBase: 001ae002 ObjectTable: fffff8583b92dfe00 HandleCount: 3190.
  Image: System
  VadRoot fffff818226674f90 Vads 5 Clone 0 Private 22. Modified 858121. Locked 0.
  DeviceMap fffff8583b9210010
  Token fffff8583b92ae040
  ElapsedTime 03:21:06.407
  UserTime 00:00:00.000
  KernelTime 00:00:42.250
  QuotaPoolUsage[PagedPool] 0
  QuotaPoolUsage[NonPagedPool] 272
  Working Set Sizes (now,min,max) (61, 50, 450) (244KB, 200KB, 1800KB)
  PeakWorkingSetSize 234
  VirtualSize 4 Mb
  PeakVirtualSize 2097156 Mb
  PageFaultCount 3326
  MemoryPriority BACKGROUND
  BasePriority 8
  CommitCharge 11

  THREAD fffff81822673a080 Cid 0004.000c Teb: 0000000000000000 Win32Thread: 0000000000000000 WAIT: (Executive) Kernel!
    fffff8047d907d20 SynchronizationEvent

  THREAD fffff81822678f080 Cid 0004.0010 Teb: 0000000000000000 Win32Thread: 0000000000000000 WAIT: (Executive) Kernel!
    fffff8047d90aa60 Semaphore Limit 0x7fffffff

  THREAD fffff8182266be080 Cid 0004.0014 Teb: 0000000000000000 Win32Thread: 0000000000000000 WAIT: (Executive) Kernel!
    fffff8047d90aa60 Semaphore Limit 0x7fffffff

```

Figure 11: Enter Caption

Name	PID	CPU	I/O total ...	Private b...	User name	Description
System Idle Process	0	7.96		60 kB	NT AUTHORITY\SYSTEM	
System	4	31.27	68 kB/s	44 kB	NT AUTHORITY\SYSTEM	NT Kernel & System
smss.exe	488			1.13 MB		Windows Session Manager
Memory Compression	2364	0.03		332 kB		
Interrupts		13.79		0		Interrupts and DPCs
Registry	128			6.67 MB		

Figure 12: Enter Caption

Please note the CID of the process = 04, parent being 00. In image 12 we can see these values correspond to Parent **System Idle Process** and Child **System**. In this listing only data about System process is displayed, even though Process Hacker and Edge are running in the background.

1.8 !process 0 0

This one is different. This command (**Purpose:**) lists all running processes, no matter if they are parents or children.

The attributes represent:

- (first) 0 – uses first syntax (see 1.7, marking (1); no ImageName defined);
- (second) 0 – sets the flags to 0, meaning minimal amount of information displayed.

Execution:

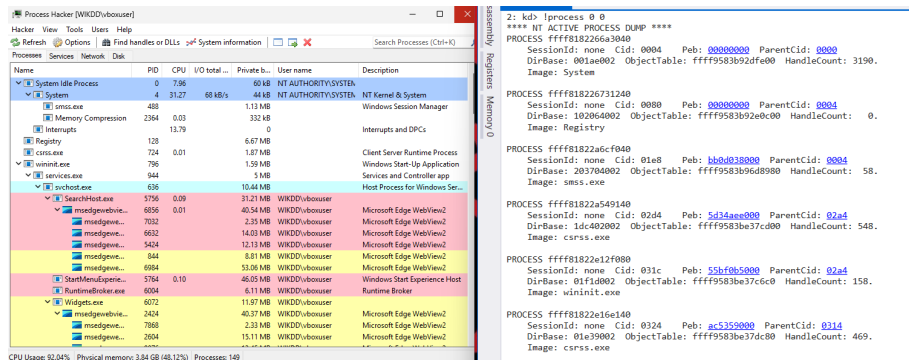


Figure 13: Right-hand side listing matches, not exactly in the same order, the listing of process hacker.

1.9 !process -1 7

This commands sets the following attributes:

1. -1 – uses the first syntax: (see 1.7, marking (1); again, no ImageName). Sets the Process address to -1, meaning that the current process is selected.
2. 7 – “To display full details on one process, set Flags to 7”. 7 is a decimal value, meaning 0x00111, setting only bits 0, 1, 2. 3rd and 4th bits are 0.

Execution:

```
2: kd> !process -1 7
PROCESS ffffff8b182266a3040
  SessionId: none Cid: 0004 Peb: 00000000 ParentId: 0000
  DirBase: 001ae002 ObjectTable: ffffff8b182266a3040 HandleCount: 3190.
  Image: System
  VadRoot ffffff8b182266a3040 Vads 5 Clone 0 Private 22. Modified 858121. Locked 0.
  DeviceMap ffffff8b182266a3040
  Token ffffff8b182266a3040
  ElapsedTime 03:21:06.407
  UserTime 00:00:00.000
  KernelTime 00:00:42.250
  QuotaPoolUsage[PagedPool] 0
  QuotaPoolUsage[NonPagedPool] 272
  Working Set Sizes (now,min,max) (64, 50, 450) (244KB, 200KB, 1800KB)
  PeakWorkingSetSize 234
  VirtualSize 4 Mb
  PeakVirtualSize 2097156 Mb
  PageFaultCount 3326
  MemoryPriority BACKGROUND
  BasePriority 8
  CommitCharge 11

  THREAD ffffff8b1822673a080 Cid 0004.000c Peb: 00000000 Win32Thread: 0000000000000000 WAIT: (Executive) KernelMode Non-Alertable
  ffffff8b1822673a080 SynchronizationEvent
  Not impersonating
  DeviceMap ffffff8b182266a3040
  Owning Process ffffff8b182266a3040 Image: System
  Attached Process N/A Image: N/A
  Wait Start TickCount 114503 Ticks: 2174 (0:00:00:33.960)
  Context Switch Count 16 IdealProcessor: 0
  UserTime 00:00:00.000
  KernelTime 00:00:00.000
  Win32 Start Address nt!PopIrpWorkerControl (0xfffff8b1822673a080)
  Stack Init ffffff8b182266a3040 Current ffffff8b182266a3040
  Base ffffff8b182266a3040 Limit ffffff8b182266a3040 Call 0000000000000000
  Priority 15 BasePriority 13 Unusual Boost 15 TopPriority 2 PagePriority 5
  Child-SP RetAddr : Args to Child : Call Site
  ffffff8b182266a3040 ffffff8b1822673a080 : 00000000 00000004 ffffff8b1822673a080 00000019 f2b49b15 00000000 00000000 : nt!KiSwapContext+0x76
  ffffff8b182266a3040 ffffff8b1822673a080 : ffffff8b1822673a080 ffffff8b1822673a080 ffffff8b1822673a080 4820244c 8b480005 : nt!KiSwapThread+0xb0
  ffffff8b182266a3040 ffffff8b1822673a080 : 74c08000 00000000 00000000 c7480005 00000000 00000000 : nt!KiCommitThreadWait+0x30d
  ffffff8b182266a3040 ffffff8b1822673a080 : ffffff8b1822673a080 ffffff8b1822673a080 ffffff8b1822673a080 83483824 4c000000 : nt!KeWaitForSingleObject+0x71f
  ffffff8b182266a3040 ffffff8b1822673a080 : ffffff8b1822673a080 ffffff8b1822673a080 ffffff8b1822673a080 be0a15ff 485474c0 84000044 : nt!PopIrpWorkerControl+0x22
  ffffff8b182266a3040 ffffff8b1822673a080 : ffffff8b1822673a080 ffffff8b1822673a080 ffffff8b1822673a080 8348c000 001cb805 : nt!PspSystemThreadStartup+0x5a
```

Figure 14: Enter Caption

1.10 !thread

This is an extension.

Purpose: Displays summary information about a thread on the target system, including the ETHREAD block. This command can be used only during kernel-mode debugging.

Syntax:

!thread [-p] [-t] [Address [Flags]]

Flavours:

-p – Displays summary information about the **process that owns the thread**.

-t – When this option is included, Address is the thread ID, not the thread address.

Address – Specifies the hexadecimal address of the thread on the target computer. If Address is -1 or omitted, it indicates the current thread.

Flags – Specifies the level of detail to display. Flags can be any combination of the following bits. **If Flags is 0, only a minimal amount of information is displayed. The default is 0x6:**

Bit 1 (0x2) – Displays the thread's wait states.

Bit 2 (0x4) – If this bit is used without Bit 1 (0x2), it has no effect. If this bit is used with Bit 1, the thread is displayed with a stack trace.

Bit 3 (0x8) – Adds the return address, the stack pointer, and (on Itanium systems) the bsp register value to the information displayed for each function and suppresses the display of function arguments.

Bit 4 (0x10) – Sets the process context equal to the process that owns the specified thread for the duration of this command. This results in more accurate display of thread stacks.

Execution:

```
2: kd> !thread
THREAD fffff81822a6be040 Cid 0004.0208 Tcb: 0000000000000000 Win32Thread: 0000000000000000 RUNNING on processor 2
Not impersonating
DeviceMap fffff9583b9210010
Owning Process fffff8182266a3040 Image: System
Attached Process N/A Image: N/A
Wait Start TickCount 116676 Ticks: 1 (0:00:00.015)
Context Switch Count 743646 IdealProcessor: 2
UserTime 00:00:00.000
KernelTime 00:07:28.750
Win32 Start Address 0xfffff80413418be0
Stack Init fffff81895059cc70 Current fffff81895059c3d0
Base fffff81895059d000 Limit fffff818950597000 Call 0000000000000000
Priority 8 BasePriority 8 IoPriority 2 PagePriority 5
Child-SP RetAddr : Args to Child : Call Site
ffffc189'5059a338 fffff804'12f614a9 : fffff8182'2a76f6b0 fffff8182'2a76f770 fffff8182'2a7ea820 00000000'00000000 : nt!DbgBreakPointWithStatus+0x1
ffffc189'5059a340 fffff8182'2a76f6b0 : fffff8182'2a76f770 fffff8182'2a7ea820 00000000'00000000 fffff8182'300f1780 : 0xfffff804'12f614a9
ffffc189'5059a348 fffff8182'2a76f770 : fffff8182'2a7ea820 00000000'00000000 fffff8182'300f1780 fffff8182'2ae83760 : 0xfffff8182'2a76f6b0
ffffc189'5059a350 fffff8182'2a7ea820 : 00000000'00000000 fffff8182'300f1780 fffff8182'2ae83760 fffff8182'266d45a0 : 0xfffff8182'2a76f770
ffffc189'5059a358 00000000'00000000 : fffff8182'300f1780 fffff8182'2ae83760 fffff8182'266d45a0 00000000'00000000 : 0xfffff8182'2a7ea820
```

Figure 15: Enter Caption

1.11 !stacks

Purpose: The !stacks extension displays information about the kernel stacks.

Syntax:

!stacks [Detail [FilterString]]
Flavours:

Detail

Specifies the level of detail to use in the display. The following table lists the valid values for *Detail*.

[Expand table](#)

0	Displays a summary of the current kernel stacks. This is the default value.
1	Displays stacks that are currently paged out, as well as the current kernel stacks.
2	Displays the full parameters for all stacks, as well as stacks that are currently paged out and the current kernel stacks.

FilterString

Displays only threads that contain the specified substring in a symbol.

Figure 16: Enter Caption

Execution:

```
2: kd> !stacks
Proc.Thread .Thread Ticks ThreadState Blocker

Max cache size is      : 1048576 bytes (0x400 KB)
Total memory in cache  : 0 bytes (0 KB)
Number of regions cached: 0
0 full reads broken into 0 partial reads
  counts: 0 cached/0 uncached, 0.00% cached
  bytes : 0 cached/0 uncached, 0.00% cached
** Prototype PTEs are implicitly decoded
    [fffff8047d9d1640 001c7c5 Idle]
0.000000 fffff8047d9d1640 001c7c5 RUNNING nt!KiIdleLoop+0x127
0.000020 fffff8182267c2080 001c7c5 STANDBY nt!KiExecuteDpcDelegate+0x90
0.000024 fffff8182267d080 001c7c5 INITIALIZEDnt!KiStartSystemThread
0.000000 fffff8182267d3080 001c7c5 RUNNING nt!KiIdleLoop+0x127
0.000028 fffff8182268202c0 001c7c5 STANDBY nt!KiExecuteDpcDelegate+0x90
0.00002c fffff8182267e4080 001c7c5 INITIALIZEDnt!KiStartSystemThread
0.000000 fffff818226828280 001c7c5 RUNNING nt!KiIdleLoop+0x127
0.000030 fffff8182268332c0 001c7c5 STANDBY nt!KiExecuteDpcDelegate+0x90
0.000034 fffff818226802080 001c7c5 INITIALIZEDnt!KiStartSystemThread
0.000000 fffff818226839280 001c7c5 RUNNING nt!KiIdleLoop+0x127
0.000038 fffff8182268442c0 001c7c5 STANDBY nt!KiExecuteDpcDelegate+0x90
0.00003c fffff818226824080 001c7c5 INITIALIZEDnt!KiStartSystemThread
    [fffff8182266a3040 System]
4.000018 fffff81822677e080 000ad88 Blocked nt!PopFxFxProcessWorkPool+0xef
4.00001c fffff8182267b1080 000001d Blocked nt!PopFxFxProcessWorkPool+0xef
4.000040 fffff81822684a280 0000e66 Blocked nt!ExpWorkQueueManagerThread+0x139
4.000044 fffff818226835080 000003c Blocked nt!KeRemovePriQueue+0x21b
4.000048 fffff818226846080 0000014 Blocked nt!ExpWorkerFactoryManagerThread+0x56
4.000064 fffff81822671c080 0000608 Blocked nt!MiRebuildLargePagesThread+0x1ad
4.000068 fffff81822671e080 001c772 Blocked nt!MiZeroPageThread+0x165
4.000070 fffff81822672b080 0000001 Blocked nt!MiGatherMappedPages+0x71
4.000074 fffff81822672d080 001b4ba Blocked nt!MiZeroNodeConductorWait+0xb9
4.00007c fffff81822672f080 0000001 READY nt!KiDispatchInterrupt+0x194
```

Figure 17: Small snippet of what is shown (current kernel stack, **Detail = 0** is default)

1.12 !stacks 2

Having attribute Detail set on 2, this displays the full parameter for all stacks (both active and paged out).

Execution:

```
2: kd> !stacks 2
Proc.Thread .Thread Ticks ThreadState Blocker

Max cache size is      : 1048576 bytes (0x400 KB)
Total memory in cache  : 0 bytes (0 KB)
Number of regions cached: 0
0 full reads broken into 0 partial reads
  counts: 0 cached/0 uncached, 0.00% cached
  bytes : 0 cached/0 uncached, 0.00% cached
** Prototype PTEs are implicitly decoded
    [fffff8047d9cef80 Idle]
0.000000 fffff8047d9d1640 001c7c5 RUNNING nt!KiIdleLoop+0x127
0.000020 fffff8182267c2080 001c7c5 STANDBY nt!KiSwapContext+0x76
    nt!KiSwapThread+0x6b0
    nt!KiExecuteDpcDelegate+0x90
    nt!KiStartSystemThread+0x34
0.000024 fffff81822676d080 001c7c5 INITIALIZEDnt!KiStartSystemThread
0.000000 fffff8182267d3080 001c7c5 RUNNING nt!KiIdleLoop+0x127
0.000028 fffff8182268202c0 001c7c5 STANDBY nt!KiSwapContext+0x76
    nt!KiSwapThread+0x6b0
    nt!KiExecuteDpcDelegate+0x90
    nt!KiStartSystemThread+0x34
0.00002c fffff8182267e4080 001c7c5 INITIALIZEDnt!KiStartSystemThread
0.000000 fffff818226828280 001c7c5 RUNNING nt!KiIdleLoop+0x127
0.000030 fffff8182268332c0 001c7c5 STANDBY nt!KiSwapContext+0x76
    nt!KiSwapThread+0x6b0
    nt!KiExecuteDpcDelegate+0x90
    nt!KiStartSystemThread+0x34
0.000034 fffff818226802080 001c7c5 INITIALIZEDnt!KiStartSystemThread
0.000000 fffff818226839280 001c7c5 RUNNING nt!KiIdleLoop+0x127
0.000038 fffff8182268442c0 001c7c5 STANDBY nt!KiSwapContext+0x76
    nt!KiSwapThread+0x6b0
    nt!KiExecuteDpcDelegate+0x90
    nt!KiStartSystemThread+0x34
```

Figure 18: Enter Caption

There are some **extra commands** worth being added to the cheat sheet:

1.13 `g`

If the process has been broken (break) via WinDBG, you can stop debugging and let the process resume by pressing this key.

1.14 `.cls`

This command clears WinDBG command window (it's like `clear` in bash terminals).

A cool resource for starting out with WinDBG: <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/getting-started-with-windbg>